COSC204 Computer Systems

Lecture 23: I/O Systems

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture: virtual memory

- Swapping and virtual memory
- Demand paging
- Page replacement algorithms
- Frame allocation
- Thrashing

This lecture: input/output (I/O) systems

- Hardware: ports, buses, controllers
- Kernel I/O interface
- I/O performance

Role of I/O devices within an OS

We've already previously covered aspects of I/O devices:

- within process scheduling (e.g., blocking on I/O),
- within device drivers (e.g., character versus block devices).

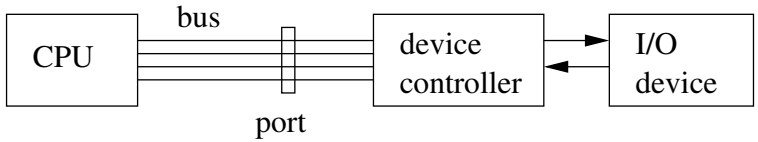
The central issues:

- How are I/O commands actually implemented?
- How does the OS manage the wide range of I/O devices that are available?
  - SSDs, spinning disks, optical drives, tape drives...
  - Network cards, models...
  - Screens, touchscreens, keyboards, printers, touchpads, mice, joysticks...

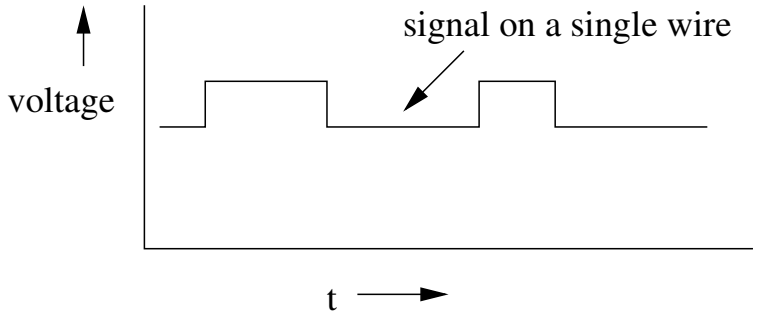
Hardware I/O: some background

An I/O device is physically linked to a computer via a port.

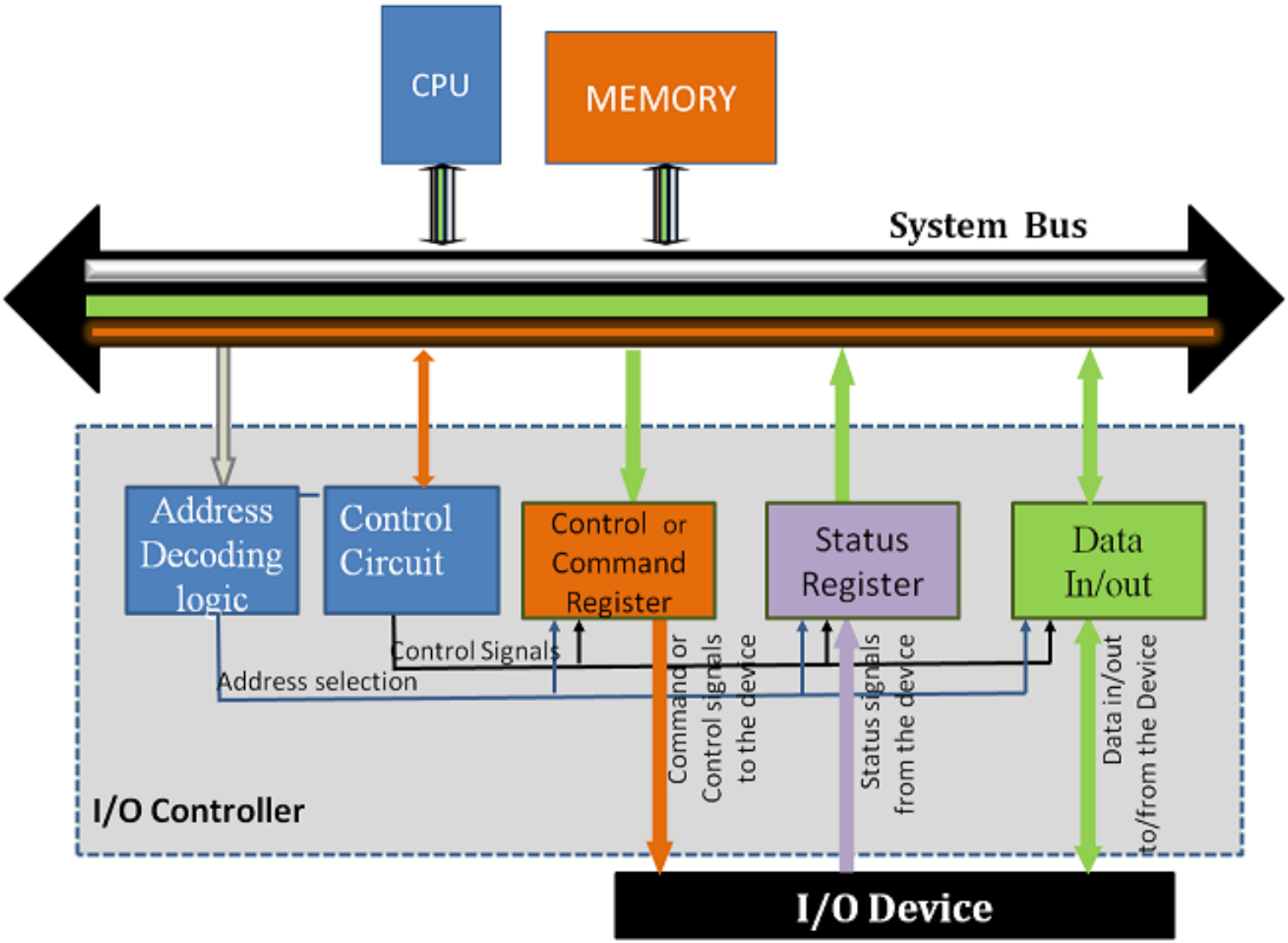
- The link is normally effected using a set of wires called a bus.
  - (. . . these days most groups of port wires aren't actually a bus, but Dave will try to go along with this oversimplification.)
- As seen before, each end of the link has a device controller.



Data transfer along a bus involves a temporal sequence of electrical voltages for each wire.

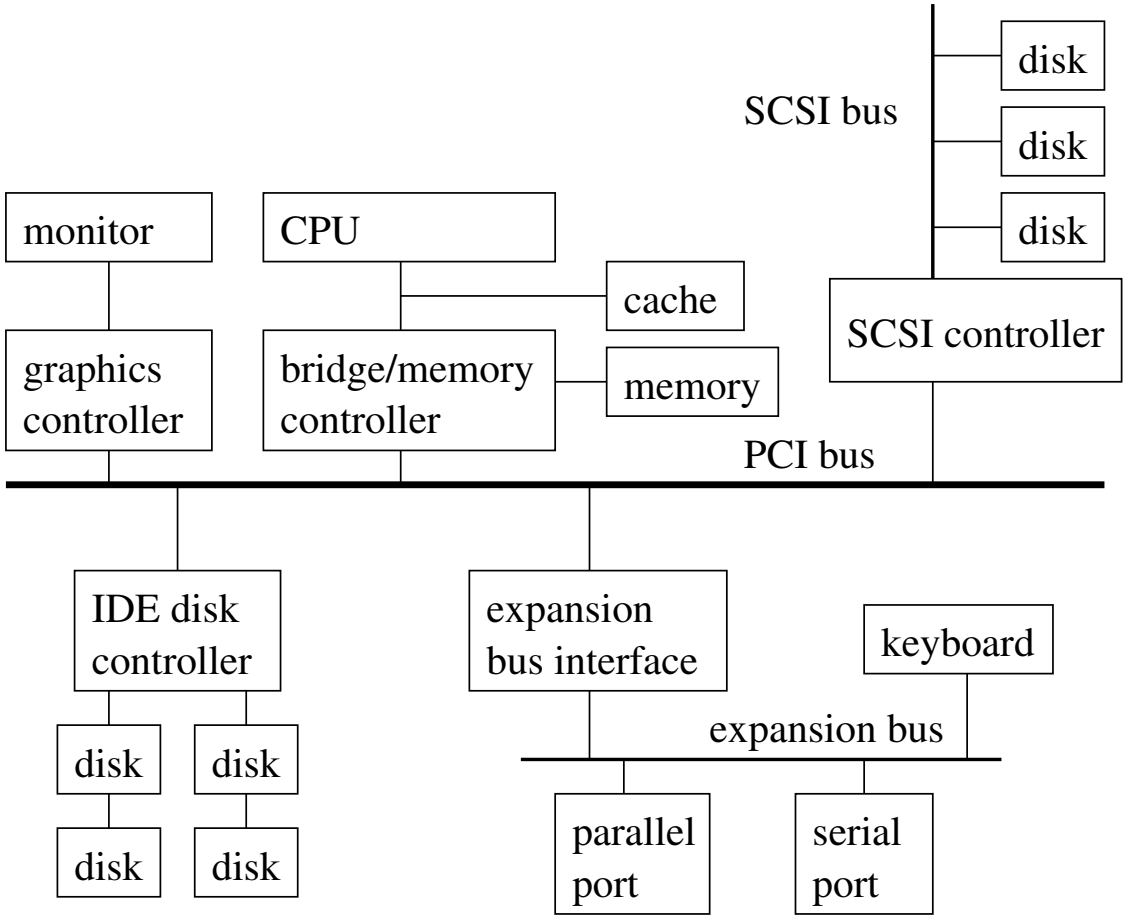


I/O controller aggregates computer's device controllers



PCI bus: a more complex (but historical) picture

On an old PC, the PCI bus might include the following components. (PCI Express (PCIe) has replaced PCI in modern machines.)



Device controller typically has data & control registers

How does the CPU give commands to a device via a bus?

- A device controller has a number of registers for holding signals.
- The CPU can effectively write and read these.
- Some registers hold data; some hold control signals.

How can the CPU specify which device it is giving commands to?

- Each different port in the system has an address range.
- The CPU can issue I/O instructions to particular addresses.
- Communication can either be a word (4 or 8 bytes) at a time (depending on 32- or 64-bit), or via direct memory access (DMA).

Polling: one approach to detecting when data is ready

Device controllers often have a status register that may contain some sort of busy bit to indicate whether the device is ready to accept data.

Polling, or busy waiting, is a simple way for the CPU to interact with a device: the CPU repeatedly checks the status register until the busy bit is clear (i.e., not busy), then writes data to the device's data register.

- This is almost always a waste of CPU time, though. (A possible exception: on microcontrollers the CPU may run a single task.)
- If the CPU is polling for input from devices too, then all devices have to be monitored—this is likely to be a big waste of CPU time.

An alternative to polling is *interrupts*.

Interrupts allow devices to gain the CPU's attention

We explored how interrupts let us implement system calls.
They are also used for communication with I/O devices.

- The CPU has a pin (i.e., a wire) called the interrupt request line.
- The CPU senses this line after each instruction it executes.
- If a signal is detected, the CPU saves its current location and jumps to interrupt handler at the address indicated in the interrupt vector (recall that there are different interrupt vectors for different devices, exceptions, etc., collected into an interrupt vector table).

If devices can raise interrupts to communicate with the CPU, the CPU doesn't need to spend time actively polling those devices.

Maskable versus non-maskable interrupts

One way of dealing with differing degrees of urgency for the requests from different devices is to provide *two* interrupt request lines:

- maskable interrupts: device-generated interrupts and traps. These can be *disabled* while the OS is in the middle of processing a more urgent interrupt. (For example, we can disable interrupts to avoid interrupts interrupting interrupt handlers!)
- non-maskable interrupts: for signalling error conditions. Unable to be disabled.

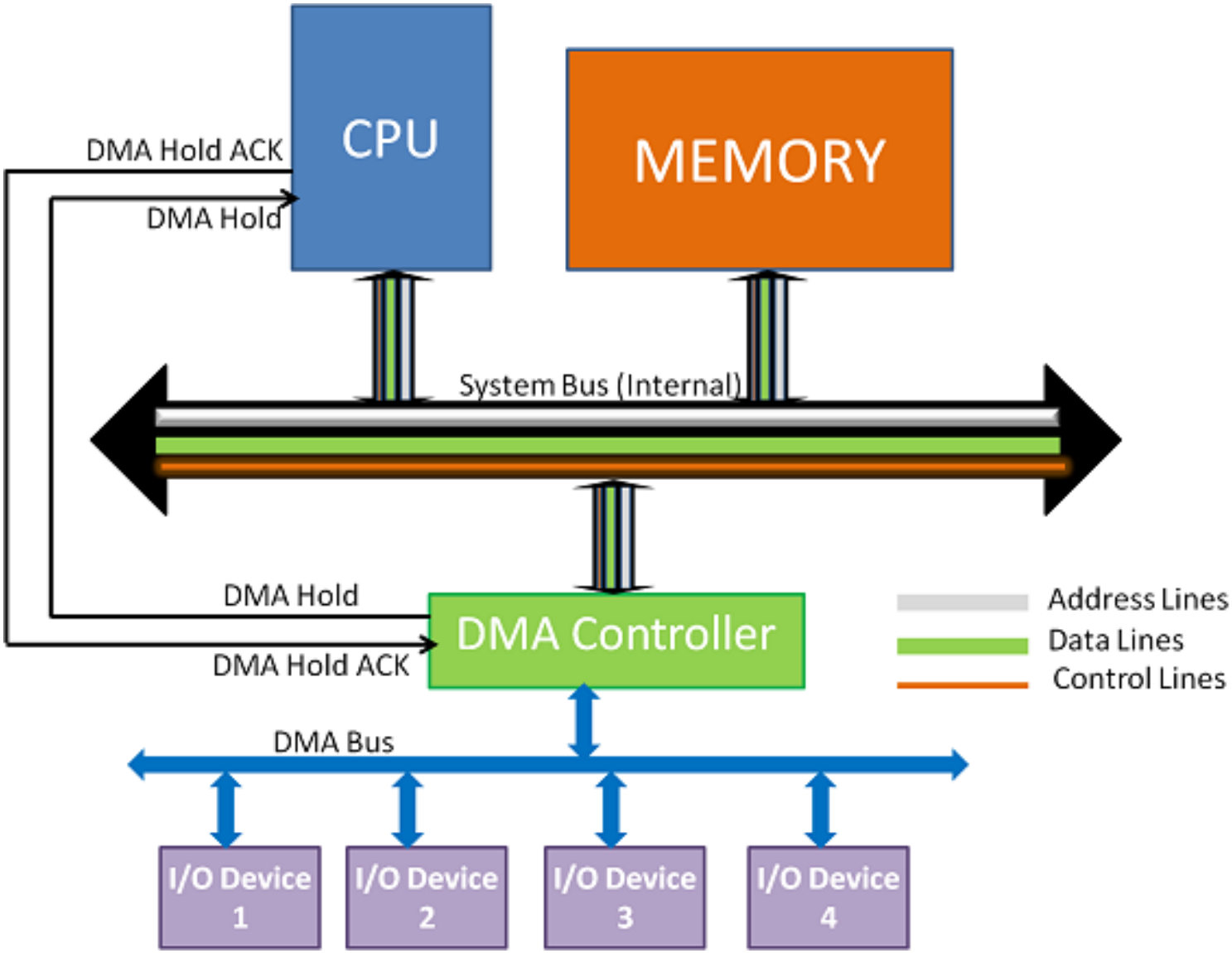
Direct Memory Access (DMA) for bulk data transfer

Programmed I/O—sending data word-by-word to a device—is laborious.

For moving large chunks of data, we can use a better method: direct memory access (DMA).

- A special-purpose processor called a DMA controller is used.
- The CPU writes a DMA command block into memory, specifying which words are to be transferred, and where to put them.
- The CPU then gives the DMA controller command block's address. The CPU and the DMA controller proceed concurrently.
- While the DMA controller is interacting with the hardware device, it controls the memory bus and moves the data between the device memory and the main memory.
- After the entire transfer is finished, the DMA controller interrupts the CPU indicating the completion of the data transfer.

Direct Memory Access (DMA) controller interactions



I/O performance is a key OS consideration

High I/O performance is critical for overall system performance. Thus, many techniques are used to optimise I/O performance, including:

- Reduce the number of context switches.
- Reduce the number of data copies in memory.
- Reduce the number of interrupts (e.g., using polling).
- Increase concurrency (e.g., DMA).
- Move data processing to hardware (e.g., network interface card).
- Balance CPU, memory system, bus, and I/O performance.

Zero-copy techniques help speed up I/O

Zero-copy approaches try to avoid memory copies being made during I/O data transfers. Such approaches are frequently used, e.g., within the network system.

- Can reduce the copying of data between the kernel and user spaces by using dynamic memory mapping.
- Can copy data from device directly to user space, e.g., Remote Direct Memory Access (RDMA).
- Can pass packet buffer pointers between layers of the TCP/IP protocol rather than copying these buffers.

Reading

For this lecture, the relevant readings include Chapter 13 (Sections 1–2, 5, 7).